Evaluating Image Processing and OCR Configurations for Chinese ID Cards

Nizar Talty Ishan Pathak

Milestone 1

Abstract

This project examines an optical character recognition (OCR) pipeline for ID card images. We design as part of this milestone a pipeline that applies optional image enhancement, selects image color channels, and extracts text and bounding boxes using either EasyOCR or Tesseract. We test the pipeline through 16 configurations that vary the aforementioned configurations. We compare them using field text recall, bounding-box location recall, normalized exact field match, and per-stage and total processing time. We adopt two testing methods: we run all of the configurations on the full dataset. The second method is we evaluate all configurations on a subset of data and then select the top 3 configurations. In the initial test on 300 images, the blue-channel EasyOCR configuration achieved the highest exact field match, 0.471, with an average end-to-end time of 0.115 seconds per image. These pilot results help identify promising configurations; the full results display the full comparison of all configurations.

1 Introduction

Optical Character Recognition (OCR) converts text in an image into machine-readable text [1]. For an ID card, the useful output includes both the text and its location on the card. This task, however, can be difficult when characters are small, image quality is degraded or not optimal, or the OCR system must recognize Chinese text [2] - the dataset provided.

The goal of this project milestone is to build a working end-to-end pipeline for Chinese ID card images and compare different ways of processing them. For this stage of the project, we focus on whether each pipeline configuration runs successfully, how accurately it extract annotated fields, and how long it takes. Future work will focus on improving the pipeline and the accuracy in cloud environments with limited computational power.

Our pipeline, as shown in Figure 1, has three pillar stages. Before loading images, pipeline configuration provided in a json file are loaded to set up all tests. The images are passed through optional image enhancement operations, color-channel selection, and finally an OCR model. An output example is shown in Figure 2.

While we do not train OCR models from scratch adapted to the dataset, we use two famous and well-established models packaged each in EasyOCR and Tesseract [3, 4]. Furthermore, we deploy the pipeline in a "staging setting" on chameleon for cloud computing and on a on-premise server to simulate edge computing. We also have two pathways to test the pipeline:

1. Running all 16 configurations defined on the full dataset. Results shown in 3.2.
2. Running all 16 configurations on a subset of 300 images, then selecting the top 3 configurations that are subsequently run on the full 20k images for final results. The results for this are in 3.1.

This was motivated by our goal of finding a more reasonable way to test the pipeline within a time budget because the full runs (16 configurations $\times$ 20k images) can take over 24 hours. Moreover, we find that using (1) and (2) yields comparable ranking of the top 3 best models.

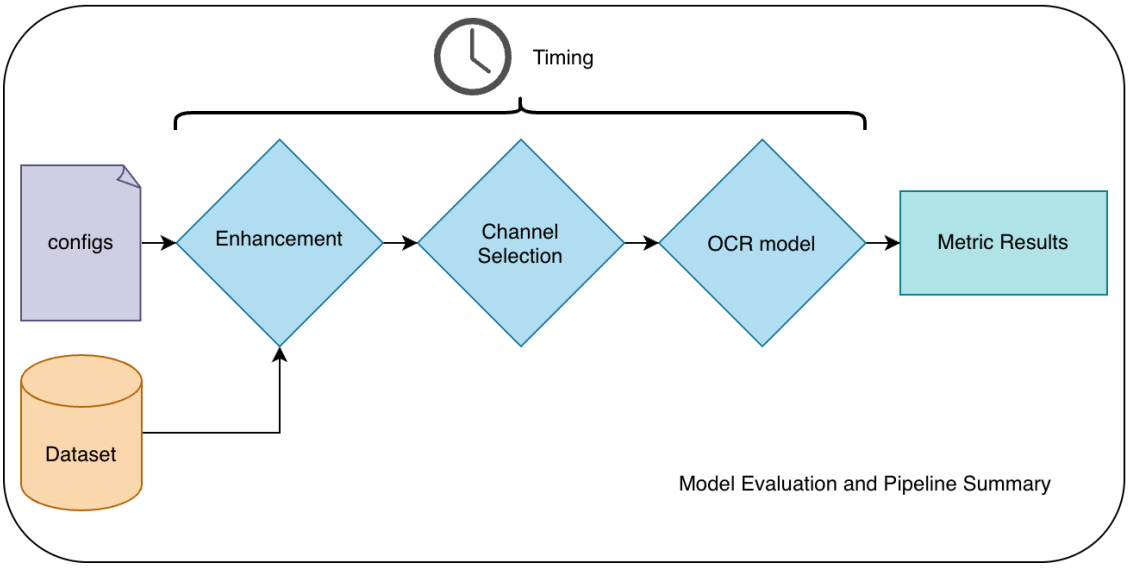


Figure 1: Overview of the ID card OCR pipeline. The pipeline starts with loading the configurations that sets the pipeline steps. An input image then passes through image enhancement, channel selection, and OCR before its output is evaluated against the annotated fields.

2 Methods

2.1 Dataset, stages, and metrics

We use the `lansinuote/ocr_id_card` dataset which contains 20,000 Chinese ID card images. Each example contains an image and annotations for its fields. An annotation gives the text of a field, the bounding boxes, and class. There are 8 classes: name, gender, ethnicity, birth year, birth month, birth day, address, and ID number.

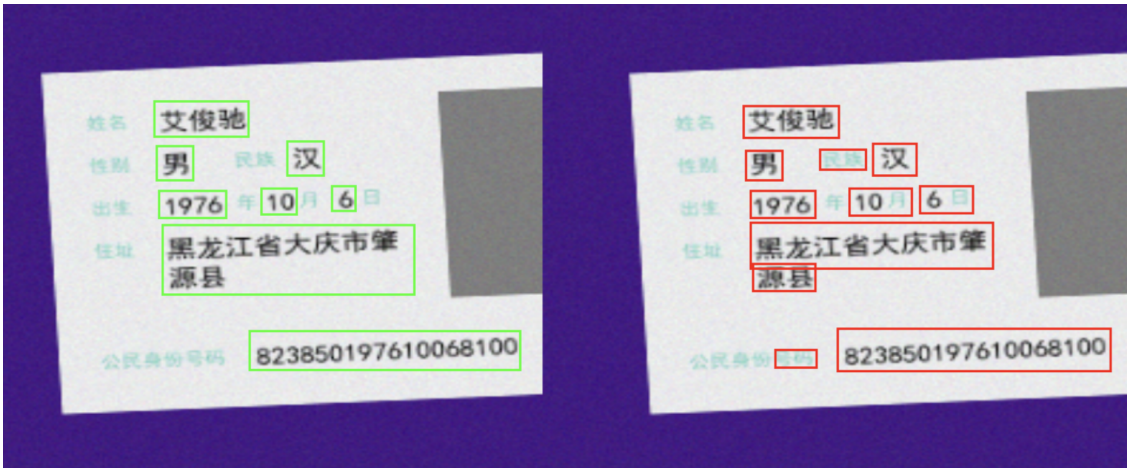


Figure 2: Example of an output of No Enhancement + Blue Channel + EasyOCR pipeline (RIGHT) compared with the ground truth annotations (LEFT).

Each configuration defines three stages. The first stages to an RGB array and optionally applies some enhancement operation (e.g., brightness, contrast, sharpening). A configuration could contain multiple enhancements that are applied in the specified order. The second stage selects the original RGB channel, one color channel, or a combination of channels. Selecting one channel gives a two-dimensional image containing that channel's values. When multiple channels are selected, the remaining channels are set to zero. The third and final stage performs OCR on the processed image. For the latter stage, we compare two OCR engines - EasyOCR and Tesseract. Both of these engines produce recognized text and boundary boxes outputs. We represent each detection by its text, box, and confidence value.

In our configuration file, we specify eight image-processing choices, each paired with both OCR engines, thus, giving 16 configurations. This can be extended in the future by simply dropping in more pre-processing configurations. The eight choices are the original image; the red, green, or blue channel alone; increased brightness with the red channel; increased contrast with the red and green channels; sharpening with the blue channel; and brightness followed by contrast on the original image. The brightness increase is 20 pixel-value units, the contrast factor is 1.2, and the sharpening amount is 1.0.

We evaluate annotated fields whose text is nonempty using different metrics. Let N be the total number of these fields across the images being evaluated. We report three field-level metrics:

$$R_{\text{text}} = \frac{N_{\text{text}}}{N}, \qquad R_{\text{location}} = \frac{N_{\text{location}}}{N}, \qquad R_{\text{exact}} = \frac{N_{\text{exact}}}{N}.$$

This means that a field contributes to N_{text} when its annotated value appears anywhere in the combined OCR text after whitespace is removed. This measure checks whether the value was recognized, regardless of its actual position. Because it uses sub-string matching, a short value may match by chance. This is a shortcoming that can be improved in later stages of the project.

To calculate N_{location}, we associate OCR detections with annotated field using their bounding boxes. A detection is eligible for a field when their overlap covers $\geq 50\%$ of the smaller box. If several

fields are eligible, the detection is assigned to the best match. we combine the boxes assigned to each field into one enclosing box. Hence, the field contributes to N_{location} when the intersection over union (IoU) of this box and the annotated box is at least 0.3.

A field contributes to N_{exact} only if it passes the location check and its assigned OCR text matches the annotated text. This is a stricter metric than N_{text}.

Additionally, we report mean OCR confidence over detections with valid confidence values. Tesseract's confidence values are divided by 100 so both engines' reported values lie between 0 and 1 (EasyOCR outputs value between 0 and 1 by default). This confidence reflects an engine's own estimate for its detections; it is not a measure of correctness, and values from the two engines are not necessarily calibrated alike.

2.2 Speeding up Tesseract and EasyOCR

Tesseract cannot use a GPU and reads one image at a time, which in our case, to run our eight Tesseract configurations on all 20,000 images in a single process would have taken about 14 hours.

That why we use multiprocessing:

We split the work across 40 CPU processes using Python's `multiprocessing.Pool`. The dataset is loaded once, and all 40 workers share it instead of each loading a copy.

- Each worker gets 50 images at a time until all 20,000 are done.
- Each worker takes an image through the whole pipeline and reports back how many fields it got right.
- The main process adds up those counts to compute the recall.

As a result, Tesseract went from about 3 images per second to approximately 26 to 40 imgs/second, roughly 8 to 11 times faster. All eight configurations finished in 1.4 hours.

It is also worth noting that we use GPU for EasyOCR as the CPU runs can be very slow explaining the allocation mentioned in 2.3. This allowed us to move from approximately 2-3 images per second to as much as 8 images per second.

2.3 Computing resources

For the full evlauation, we used one Chameleon `g1.h100.pci.1` instance with an NVIDIA H100 GPU (94 GB of memory), 48 vcpus, 244GB of RAM, and 40GB of disk space.

For the subset benchmark where we test on 300 images then only run full dataset on the top-3 best performing configurations, we used a machine equipped with NVIDIA RTX PRO 6000 Blackwell

(98 GB of vRAM) with 72 vCPUs. We chose not to do it on Chameleon in order to save on Chameleon resources (and "simulate" some on-premise edge server).

3 Results

We report for this milestone text recall, location recall, normalized exact field match, average OCR confidence, and processing time. We rank configurations by exact field match because it requires both matching text and a matching location.

3.1 Evaluation on a subset

We first evaluated all 16 configurations on the same randomly selected set of 300 images. Table 1 summarizes the results. Blue-channel EasyOCR "recipe" hadthe highest exact field match at 0.471. Sharpened-blue EasyOCR ranked second at 0.447, followed by green-channel EasyOCR at 0.438. We selected these three configurations for a separate run on the full dataset.

Image processing	Engine	Text	Location	Exact	Conf.	s/image	Total (s)
Blue	**EasyOCR**	**0.735**	**0.791**	**0.471**	0.520	0.115	34.5
Sharpen blue	EasyOCR	0.718	0.774	0.447	0.501	0.112	33.6
Green	EasyOCR	0.743	0.763	0.438	0.461	0.127	38.1
Bright+contrast original	EasyOCR	0.725	0.723	0.366	0.426	0.144	43.1
Original	EasyOCR	0.750	0.695	0.346	0.374	0.128	38.5
Contrast red+green	EasyOCR	0.752	0.672	0.325	0.374	0.144	43.2
Blue	Tesseract	0.508	0.657	0.318	0.654	0.240	72.0
Sharpen blue	Tesseract	0.516	0.678	0.315	0.625	0.238	71.5
Red	EasyOCR	0.754	0.694	0.305	0.337	0.141	42.3
Bright red	EasyOCR	0.746	0.688	0.304	0.335	0.132	39.6
Bright+contrast original	Tesseract	0.515	0.672	0.288	0.601	0.286	85.8
Green	Tesseract	0.497	0.607	0.282	0.642	0.246	73.9
Original	Tesseract	0.514	0.690	0.264	0.600	0.326	97.9
Contrast red+green	Tesseract	0.501	0.645	0.250	0.590	0.305	91.4
Bright red	Tesseract	0.496	0.640	0.241	0.589	0.264	79.1
Red	Tesseract	0.484	0.647	0.236	0.593	0.272	81.6

Table 1: Results for all 16 configurations on the same 300-image evaluation set, ordered by exact field match. **Important**: Tesseract does not use multiprocessing here.

One interesting note, Table 2 shows the time for each stage of the pipeline. With no surprise, OCR is the stage that sets the pace: for blue-channel EasyOCR, it accounted for 32.1 of the 34.5 total seconds. The total also includes dataset access and loop overhead, so it could exceed the sum of the measured stages.

	Seconds per image				Total seconds			
Image processing	Prep.	OCR	Post.	Overall	Prep.	OCR	Post.	Overall
Blue	0.001	0.107	0.001	0.115	0.2	32.1	0.2	34.5
Sharpen blue	0.001	0.104	0.001	0.112	0.4	31.1	0.2	33.6
Green	0.001	0.119	0.001	0.127	0.2	35.6	0.2	38.1

Table 2: 300-images subset timing breakdown for the three selected EasyOCR configurations.

3.2 Full Evaluation

After the benchmark mentioned above, we ran the three configurations selected on all 20k images. As Table 3 shows, their order by exact field match remained blue, sharpened blue, then green. Blue-channel EasyOCR achieved 0.492 exact field match and 0.805 location recall.

Image processing	Text	Location	Exact	Conf.	s/image	Total (s)
Blue	**0.746**	**0.805**	**0.492**	0.539	0.111	2225.8
Sharpen blue	0.727	0.792	0.466	0.512	0.114	2285.1
Green	0.749	0.766	0.430	0.451	0.121	2419.6

Table 3: Separate full-dataset run of the three configurations selected by the pilot. Each row uses 20,000 images.

On Chameleon instance, we evaluated all 16 configurations on the full dataset. Table 4 shows that the best Tesseract configuration was sharpened blue with an exact field match of 0.310 lagging behind the best OCR pipeline.

It is worth noting that the small benchmark (16 configs on 300 imgs then top-3 on 20k imgs) retained the same order when compared with every configuration on 20k images. This shows that in real settings, under time constraints and/or budgets, one needs to be more sensible to how benchmarks are conducted so as to make the most of the available computational power with limited time.

4 Discussion

Blue-Channel EasyOCR gave the strongest overall result on the full dataset: 0.492 exact field match, compared with 0.346 for EasyOCR on the original image. This suggests that channel selection affected how well OCR detections matched the annotated fields. Sharpening the blue channel did not improve the result; its exact field match was 0.466.

Furthermore, the different accuracy measures show why we should not judge only by text recall. Red-channel EasyOCR had slighlty higher text recall than blue-channel on the full dataset (0.758 versus 0.746), but considerably lower exact field match (0.305 versus 0.492). Text recall only checks whether a value appears somewhere in the OCR output. Exact field match also requires a matching location. The 0.492 score means that 49.2% of annotated fields passed this stricter check; however, it does not mean that 49.2% of entire cards were read perfectly.

Throughout this first step of the project, we have a few limitation to address:

Image processing	Engine	Text	Location	Exact	Conf.	s/image	Total (s)
Blue	**EasyOCR**	**0.746**	**0.805**	**0.492**	0.539	0.118	2353.4
Sharpen blue	EasyOCR	0.727	0.792	0.466	0.512	0.121	2412.9
Green	EasyOCR	0.749	0.766	0.430	0.451	0.128	2562.3
Bright+contrast original	EasyOCR	0.738	0.717	0.365	0.421	0.146	2924.2
Original	EasyOCR	0.754	0.693	0.346	0.377	0.142	2847.4
Contrast red+green	EasyOCR	0.757	0.671	0.320	0.365	0.152	3039.9
Sharpen blue	Tesseract	0.535	0.694	0.310	0.636	0.030	593.1
Red	EasyOCR	0.758	0.702	0.305	0.334	0.151	3028.7
Blue	Tesseract	0.524	0.650	0.303	0.663	0.025	504.4
Bright red	EasyOCR	0.751	0.699	0.302	0.332	0.154	3083.5
Green	Tesseract	0.513	0.638	0.272	0.637	0.026	524.5
Bright+contrast original	Tesseract	0.529	0.671	0.263	0.599	0.032	640.9
Original	Tesseract	0.523	0.681	0.243	0.591	0.034	680.4
Contrast red+green	Tesseract	0.516	0.665	0.241	0.588	0.035	695.5
Red	Tesseract	0.491	0.644	0.225	0.585	0.038	757.5
Bright red	Tesseract	0.494	0.638	0.224	0.583	0.039	772.6

Table 4: All 16 configurations evaluated on 20,000 images, ordered by exact field match. EasyOCR used the GPU; **Important**: Tesseract used 40 CPU processes. Tesseract's s/image is the total run time divided by the number of images while 40 processes run in parallel.

- Limit the computational power and use less "overkill" machine allocation for a more realistic setting.
- Some of the metrics can be inflated or worsened due to errors in how they are accounted for. This prompts us to have a more principled analysis and visual inspection of the pipeline results.
- Extend the configurations.

5 Teamwork and Reproducibility Statement

Work has been split up in a way to make sure there is equal contribution. Ishan and Nizar both worked on the code iterating over each other's scripts. As for writing and presenting: Nizar mostly took care of the report, while Ishan took care of the presentation deck of slides.

Reproducibility: all of the code that defines the pipeline and the scripts to run them are available in the Github Repository of our team.

6 Conclusion

We implemented and evaluated 16 ID card OCR pipeline configurations: varying image enhancement, color-channel selection, and OCR engine. Blue-channel EasyOCR ranked the best by *exact*

field match in the 300-image evaluation and in the full evaluation of 20,000 images. Its corresponding full-dataset exact field match was 0.492. The comparison shows that the image-processing choice matters and that finding field text somewhere in an image is different from matching both its text and location. Future work can improve field matching and metrics and test the pipeline visually to better identify model failures.

References

[1] T. T. H. Nguyen, A. Jatowt, M. Coustaty, and A. Doucet, "Survey of Post-OCR Processing Approaches," *ACM Computing Surveys*, vol. 54, no. 6, Article 124, 2021. doi: 10.1145/3453476.

[2] T.-L. Yuan, Z. Zhu, K. Xu, C.-J. Li, T.-J. Mu, and S.-M. Hu, "A Large Chinese Text Dataset in the Wild," *Journal of Computer Science and Technology*, vol. 34, no. 3, pp. 509–521, 2019. doi: 10.1007/s11390-019-1923-y.

[3] JaidedAI, "EasyOCR: Ready-to-use OCR with 80+ supported languages and all popular writing scripts," GitHub software repository. Available: `https://github.com/JaidedAI/EasyOCR`.

[4] R. Smith, "An Overview of the Tesseract OCR Engine," in *Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*, vol. 2, pp. 629–633, 2007. doi: 10.1109/ICDAR.2007.4376991.